

Réalisé par : Audes Mariana MOUSSAVOU MOUSSAVOU

TP 2: Sécurisation d'une communication client-serveur avec chiffrement Fernet (AES)

Objectifs pédagogiques :

- Comprendre les limites d'une communication réseau non sécurisée.
- Mettre en place un chiffrement symétrique avec Fernet.
- Observer l'impact sur le trafic avec Wireshark.
- Appliquer une sécurisation sur un scénario métier concret.

Contexte du cas d'usage :

Une entreprise développe une application client-serveur pour que les employés puissent envoyer leur numéro de badge pour valider leur présence. Les échanges passent sur le réseau interne. L'équipe sécurité soupçonne que ces messages peuvent être interceptés. Elle demande à vous, techniciens en cybersécurité, de sécuriser cette communication.

Matériel requis :

- Python 3 installé
- Bibliothèque `cryptography` installée
- Wireshark installé
- Deux terminaux (ou deux machines si possible)

Partie 1 – Mise en place d'une communication simple

But : comprendre les faiblesses du réseau en clair

1. Créez un petit programme `serveur` qui attend un message d'un client (le numéro de badge).

```

Server.py > ...
1  import socket
2
3  server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
4  server_address=('172.16.24.56',9091)
5  server_socket.bind(server_address)
6  server_socket.listen(1)
7  #attendre une connexion d'un client
8  connection,client_address=server_socket.accept()
9  while True:
10     received_msg=connection.recv(1024).decode()
11     print(f'lui :{received_msg}')
12     message=input("moi :")
13     connection.sendall(message.encode())
14     if received_msg=='bye' or message=='bye':
15         break
16

```

2. Créez un client qui se connecte au serveur et envoie le numéro de badge.

```
Client.py > ...
1  import socket
2
3  client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
4  server_address=('172.16.24.56',9091)
5  client_socket.connect(server_address)
6  while True:
7      message=input("moi:")
8      client_socket.sendall(message.encode())
9      received_msg=client_socket.recv(1024).decode()
10     print(f'lui :{received_msg}')
11     if received_msg=='bye' or message=='bye':
12         break
```

3. Exécutez le client et le serveur sur le même poste (ou deux machines).

```
PS C:\Users\22891\Desktop\Programmation distribuée et sécurité> & c:/pyth
Client:Bonjour
Serveur :Bonjour
Client:comment tu vas ?
[]
```

```
PROBLÈMES SORTIE CONSOLE DE DÉBOGAGE TERMINAL PORTS
PS C:\Users\22891\Desktop\Programmation distribuée et sécurité> & c:/Py
Client :Bonjour
Seveur :Bonjour
Client :comment tu vas ?
Seveur :[]
```

4. Lancez Wireshark pendant l'échange, et filtrez par le port utilisé.

tcp.port == 9091						
No.	Time	Source	Destination	Protocol	Length	Info
7	9.295753	172.16.24.56	172.16.24.56	TCP	56	35049 → 9091 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
8	9.295838	172.16.24.56	172.16.24.56	TCP	56	9091 → 35049 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
9	9.295869	172.16.24.56	172.16.24.56	TCP	44	35049 → 9091 [ACK] Seq=1 Ack=1 Win=65280 Len=0
11	14.037962	172.16.24.56	172.16.24.56	TCP	52	35049 → 9091 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=0 [Malformed Packet]
12	14.038006	172.16.24.56	172.16.24.56	TCP	44	9091 → 35049 [ACK] Seq=1 Ack=9 Win=65280 Len=0
14	20.358417	172.16.24.56	172.16.24.56	TCP	51	9091 → 35049 [PSH, ACK] Seq=1 Ack=9 Win=65280 Len=7
15	20.358467	172.16.24.56	172.16.24.56	TCP	44	35049 → 9091 [ACK] Seq=9 Ack=8 Win=65280 Len=0
19	28.918432	172.16.24.56	172.16.24.56	TCP	60	35049 → 9091 [PSH, ACK] Seq=9 Ack=8 Win=65280 Len=16
20	28.918481	172.16.24.56	172.16.24.56	TCP	44	9091 → 35049 [ACK] Seq=8 Ack=25 Win=65280 Len=0

5. Observez : le message (numéro de badge) est-il lisible dans les paquets capturés ?

```
Wireshark · Suivre le flux TCP (tcp.stream eq 0) · Adapter for loopback traffic capture

Bonjour
Bonjour
comment tu vas ?
```

Partie 2 – Mise en place du chiffrement avec Fernet

But : sécuriser l'échange pour empêcher toute interception compréhensible

6. Créez une **clé symétrique** partagée entre le client et le serveur (à générer une seule fois).

```
generatekey.py > ...
1  from cryptography.fernet import Fernet
2  key=Fernet.generate_key()
3  print(key)
4  with open("secret_key.key","wb") as f:
5      f.write(key)
```

```
secret_key.key
1  SZHJOaBPbGA1QMxDd7UuEFv78yaEqsJA0ggKgG211Pk=
```

7. Le client chiffre le numéro de badge avant de l'envoyer.

```
Client.py > ...
1  import socket
2  from cryptography.fernet import Fernet
3
4  client_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
5  server_address=('172.16.24.56',9091)
6
7  client_socket.connect(server_address)
8  with open("secret_key.key","rb") as f:
9      key=f.read()
10     fernet=Fernet(key)
11
12     while True:
13         message=input("moi:")
14         client_socket.sendall(fernet.encrypt(message.encode()))
15         received_msg=fernet.decrypt(client_socket.recv(1024)).decode()
16         print(f'lui :{received_msg}')
17         if received_msg=='bye' or message=='bye':
18             break
```

8. Le serveur déchiffre le message reçu

```
Server.py > ...
1  import socket
2  from cryptography.fernet import Fernet
3
4  server_socket=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
5  server_address=('172.16.24.56',9091)
6  server_socket.bind(server_address)
7  server_socket.listen(1)
8  #attendre une connexion d'un client
9  connection,client_address=server_socket.accept()
10     with open("secret_key.key","rb") as f:
11         key=f.read()
12     fernet = Fernet(key)
13
14     while True:
15         received_msg=fernet.decrypt(connection.recv(1024)).decode()
16         print(f'lui :{received_msg}')
17         message=input("moi:")
18         connection.sendall(fernet.encrypt(message.encode()))
19         if received_msg=='bye' or message=='bye':
20             break
```

9. Vérifiez que la communication fonctionne toujours.

tcp.port == 9091						
No.	Time	Source	Destination	Protocol	Length	Info
2	1.697130	172.16.24.56	172.16.24.56	TCP	44	9091 → 43451 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	12.805506	172.16.24.56	172.16.24.56	TCP	56	47277 → 9091 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
10	12.805601	172.16.24.56	172.16.24.56	TCP	56	9091 → 47277 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
11	12.805651	172.16.24.56	172.16.24.56	TCP	44	47277 → 9091 [ACK] Seq=1 Ack=1 Win=65280 Len=0
14	17.099756	172.16.24.56	172.16.24.56	TCP	144	47277 → 9091 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=100
15	17.099812	172.16.24.56	172.16.24.56	TCP	44	9091 → 47277 [ACK] Seq=1 Ack=101 Win=65280 Len=0
17	26.517876	172.16.24.56	172.16.24.56	TCP	144	9091 → 47277 [PSH, ACK] Seq=1 Ack=101 Win=65280 Len=100
18	26.517132	172.16.24.56	172.16.24.56	TCP	44	47277 → 9091 [ACK] Seq=101 Ack=101 Win=65280 Len=0
23	35.179696	172.16.24.56	172.16.24.56	TCP	164	47277 → 9091 [PSH, ACK] Seq=101 Ack=101 Win=65280 Len=120
24	35.179752	172.16.24.56	172.16.24.56	TCP	44	9091 → 47277 [ACK] Seq=101 Ack=221 Win=65280 Len=0
✓ 27	47.075937	172.16.24.56	172.16.24.56	TCP	164	9091 → 47277 [PSH, ACK] Seq=101 Ack=221 Win=65280 Len=120
28	47.075988	172.16.24.56	172.16.24.56	TCP	44	47277 → 9091 [ACK] Seq=221 Ack=221 Win=65280 Len=0

Wireshark · Suivre le flux TCP (tcp.stream eq 1) · Adapter for loopback traffic capture

gAAAAABpovnC1cM48Bfu9uYMIKGuveHjMo1jTc0apPuMkuxBs_APJYKcGU2jJd082apG3gduGKkOd471T5A4BZxwAmNvUx2PuQ==
gAAAAABpovnmPkvJNW_Fw1tTG410BStvhfnhPd-Q4Xqe_1ocsuResKu4YlzmBscYJCipIUZtbQr7M4glLmoPFA9P0yI2Ztfxw==
gAAAAABpovnUV_KlMyD3cmEIUqhH72P4px1DT7aPpwObYI1-ZKgAtwPdjrui2xnJl0k_Wh0l3t11ZNJIVjJqAIBGcgyAYyN4fPT7QrysugjopDosN2eI_w=
gAAAAABpovngAMl-wyF5vBzBmzGTt9EsyPKvj-oYzKcailp69Y64MxnN5EhCTUes6EU_PehDVRdFR-gYjorxX9aXfxXzuVKFtxmQo3cJATRj-xNBwedY01I=

Partie 3 – Analyse du trafic sécurisé

But : comparer avec la version non sécurisée

Avec la version non sécurisée, on est capable de voir les message envoyé clairement. Ce qui compromet. La confidentialité et par conséquent l'intégrité.